

Ćwiczenie laboratoryjne nr.5

Ksawery Zelek

Fragmenty kodów

Czyszczenie i przygotowanie środowiska

```
4 clear all
5 clc
```

Na początek czyścimy konsolę i pamięć, żeby nie mieszały się nam stare wyniki z poprzednich zadań.

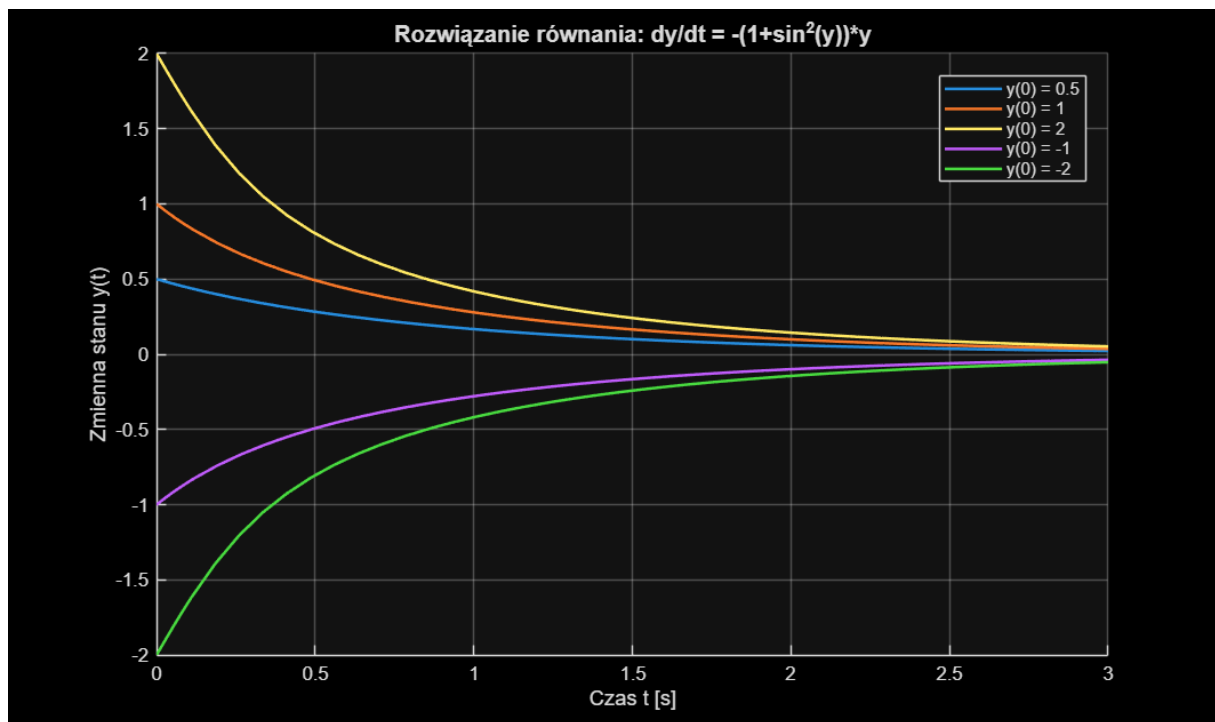
ZADANIE 1 – Równanie nieliniowe I rzędu ($\frac{dy}{dt} = -(1 + \sin^2(y))y$)

Z postaci równania wynika, że czynnik $(1+\sin^2(y))$ jest zawsze dodatni i przyjmuje wartości od **1** do **2**. Ponieważ całość jest poprzedzona znakiem minus, znak pochodnej jest zawsze przeciwny do znaku aktualnej wartości **y**. Powoduje to, że układ zachowuje się jak tłumik – każda wygenerowana trajektoria, niezależnie od tego, czy startuje z wartości dodatnich, czy ujemnych, asymptotycznie zbiega do stanu równowagi **y = 0**. Wykresy wyraźnie pokazują, że im wyższa wartość bezwzględna warunku początkowego, tym bardziej stromy jest początkowy spadek (lub wzrost) krzywej.

KOD:

```
4 % Definicja równania (funkcja anonimowa)
5 nieliniowe_rr = @(t, y) -(1 + (sin(y))^2) * y;
6
7 przedzial_czasu = [0, 3];
8 warunki_pocz = [0.5, 1, 2, -1, -2];
9
10 figure('Name', 'Zadanie 5.7', 'Color', 'w');
11 hold on; grid on;
12
13 % Wykorzystanie pętli do iteracji po warunkach początkowych
14 for idx = 1:length(warunki_pocz)
15     y0 = warunki_pocz(idx);
16     [t_out, y_out] = ode45(nieliniowe_rr, przedzial_czasu, y0);
17     plot(t_out, y_out, 'LineWidth', 1.5, 'DisplayName', sprintf('y(0) = %g', y0));
18 end
19
20 title('Rozwiązanie równania: dy/dt = -(1+sin^2(y))*y', 'FontSize', 12);
21 xlabel('Czas t [s]', 'FontSize', 11);
22 ylabel('Zmienna stanu y(t)', 'FontSize', 11);
23 legend('Location', 'best');
24 hold off;
```

WYKRES:



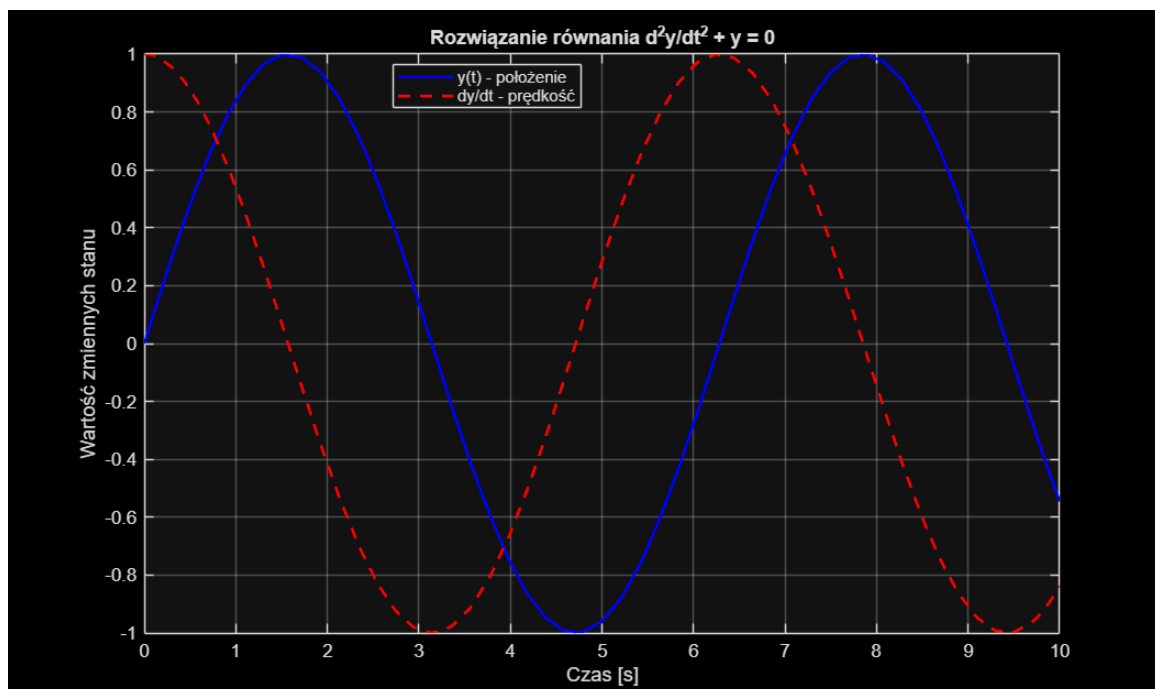
ĆWICZENIE 2 – Równanie drugiego rzędu ($\frac{d^2y}{dt^2} + y = 0$)

Powyższe równanie opisuje idealny oscylator harmoniczny (bez tłumienia). Otrzymane przebiegi to czyste funkcje trygonometryczne (sinus dla położenia i cosinus dla prędkości). Amplitudy nie maleją w czasie, a oba sygnały są przesunięte w fazie względem siebie o równe $\pi/2$ co potwierdza poprawność rozwiązania analitycznego układu zachowawczego.

KOD:

```
4      % Układ równań stanu
5      układ_oscylatora = @(t, x) [x(2); -x(1)];
6
7      warunki_startowe = [0; 1];
8      czas_symulacji = [0, 10];
9
10     [T, X] = ode45(układ_oscylatora, czas_symulacji, warunki_startowe);
11
12     figure('Color', 'black');
13     plot(T, X(:, 1), 'b-', 'LineWidth', 1.5); hold on;
14     plot(T, X(:, 2), 'r--', 'LineWidth', 1.5);
15     grid on;
16
17     title('Rozwiązanie równania  $d^2y/dt^2 + y = 0$ ');
18     xlabel('Czas [s]');
19     ylabel('Wartość zmiennych stanu');
20     legend('y(t) - położenie', 'dy/dt - prędkość', 'Location', 'best');
21     hold off;
```

WYKRES:



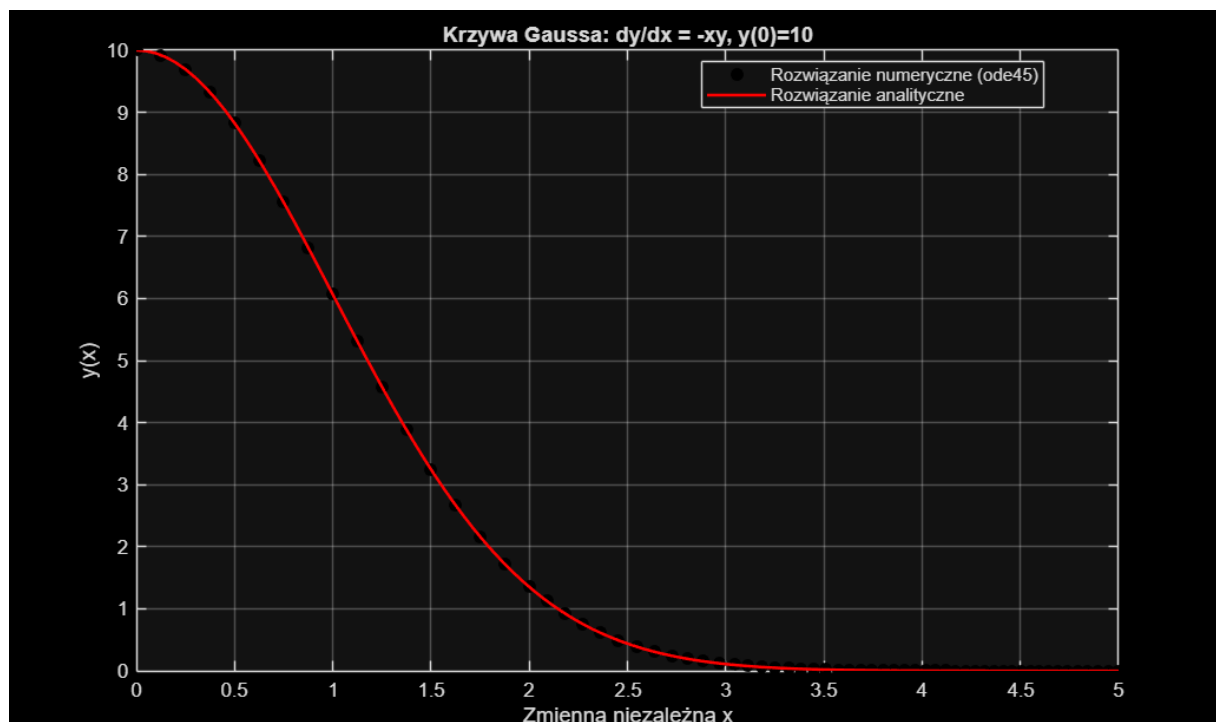
ĆWICZENIE 3 – Równanie z modelem Gaussa ($\frac{dy}{dx} = -xy$)

Rozwiązanie tego równania prowadzi do funkcji, która jest połówką klasycznej krzywej dzwonowej (krzywej Gaussa). Wykres pokazuje, że wartość funkcji bardzo szybko zanika, zbliżając się asymptotycznie do osi poziomej w okolicach $x=3$. Algorytm ode45 adaptacyjnie dobiera krok całkowania, co widać po zagęszczeniu punktów pomiarowych w rejonach, gdzie krzywa opada najbardziej stromo.

KOD:

```
4 rownanie_gaussa = @(x, y) -x * y;
5
6 [X_num, Y_num] = ode45(rownanie_gaussa, [0, 5], 10);
7
8 % Obliczenie wartości analitycznych do weryfikacji
9 X_analityczne = linspace(0, 5, 100);
10 Y_analityczne = 10 * exp(-(X_analityczne.^2) / 2);
11
12 figure('Color', 'black');
13 plot(X_num, Y_num, 'ko', 'MarkerFaceColor', 'k'); hold on;
14 plot(X_analityczne, Y_analityczne, 'r-', 'LineWidth', 1.5);
15 grid on;
16
17 title('Krzywa Gaussa: dy/dx = -xy, y(0)=10');
18 xlabel('Zmienna niezależna x');
19 ylabel('y(x)');
20 legend('Rozwiązanie numeryczne (ode45)', 'Rozwiązanie analityczne', 'Location', 'best');
21 hold off;
```

WYKRES:



ĆWICZENIE 4 – Równanie metod ($y' = -ty^2$)

Zarówno metoda ode23 (niższego rzędu), jak i ode45 (wyższego rzędu) radzą sobie z tym problemem bardzo dobrze, a ich wyniki pokrywają się z rozwiązaniem analitycznym do kilku miejsc po przecinku. Różnice między solverami pojawiają się dopiero przy dalszych cyfrach znaczących. Dla prostych i gładkich funkcji (nie wykazujących "sztywności"), użycie ode45 jest standardem gwarantującym optymalną dokładność.

KOD:

```
4 rownanie_kwadratowe = @(t, y) -2 * t * (y^2);
5 wektor_t = [0, 0.25, 0.5, 0.75, 1];
6 y0 = 1;
7
8 [~, wynik_ode23] = ode23(rownanie_kwadratowe, wektor_t, y0);
9 [~, wynik_ode45] = ode45(rownanie_kwadratowe, wektor_t, y0);
10
11 % Wynik analityczny
12 wynik_anal = 1 ./ (1 + wektor_t.^2)';
13
14 % Wyświetlenie wyników w estetycznej tabeli
15 Tabela_Porownawcza = table(wektor_t', wynik_ode23, wynik_ode45, wynik_anal, ...
16 'VariableNames', {'Czas_t', 'Metoda_ode23', 'Metoda_ode45', 'Analitycznie'})
```

WYKRES:

Tabela_Porownawcza =

5×4 [table](#)

Czas_t	Metoda_ode23	Metoda_ode45	Analitycznie
0	1	1	1
0.25	0.94118	0.94118	0.94118
0.5	0.80002	0.8	0.8
0.75	0.64002	0.64	0.64
1	0.5	0.5	0.5

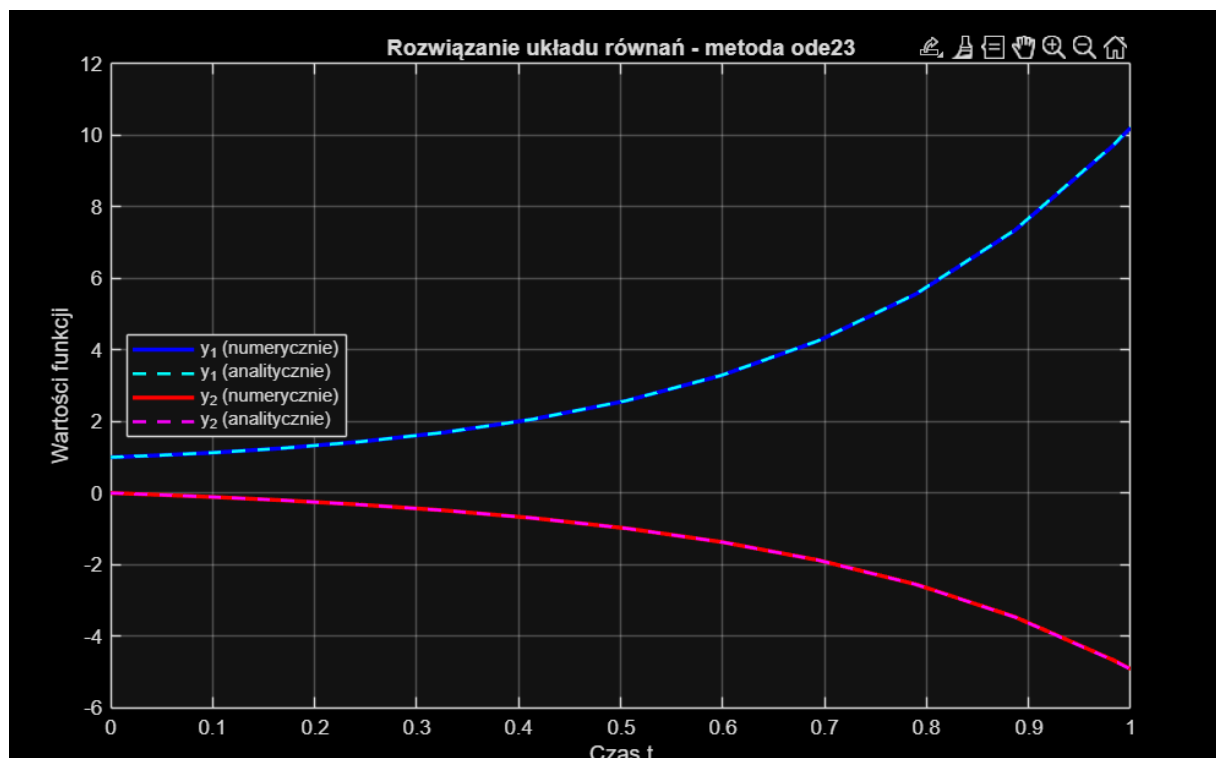
ĆWICZENIE 5 – Układ równań I rzędu

Na wygenerowanym wykresie obserwujemy idealne pokrywanie się linii przerywanych (wartości teoretyczne) z ciągłymi (wartości numeryczne). W rozwiązaniu analitycznym występuje czynnik e^{3t} . Właśnie ten czynnik powoduje, że cały układ ma charakter niestabilny (rozbieżny) – wartość y_1 zaczyna gwałtownie rosnąć do wartości dodatnich, podczas gdy y_2 dąży silnie w stronę wartości ujemnych.

KOD:

```
4 sys_rownan = @(t, y) [y(1) - 4*y(2); -y(1) + y(2)];
5 czas_t = [0, 1];
6 warunki_pocz = [1; 0];
7
8 [T_uklad, Y_uklad] = ode23(sys_rownan, czas_t, warunki_pocz);
9
10 % Wzory analityczne
11 y1_teoria = 0.5 * (exp(-T_uklad) + exp(3*T_uklad));
12 y2_teoria = 0.25 * (-exp(3*T_uklad) + exp(-T_uklad));
13
14 figure('Color', 'black');
15 plot(T_uklad, Y_uklad(:,1), 'b-', 'LineWidth', 2); hold on;
16 plot(T_uklad, y1_teoria, 'c--', 'LineWidth', 1.5);
17 plot(T_uklad, Y_uklad(:,2), 'r-', 'LineWidth', 2);
18 plot(T_uklad, y2_teoria, 'm--', 'LineWidth', 1.5);
19 grid on;
20
21 title('Rozwiązanie układu równań - metoda ode23');
22 xlabel('Czas t'); ylabel('Wartości funkcji');
23 legend('y_1 (numerycznie)', 'y_1 (analitycznie)', 'y_2 (numerycznie)', 'y_2 (analitycznie)', 'Location', 'west');
24 hold off;
```

WYKRES:



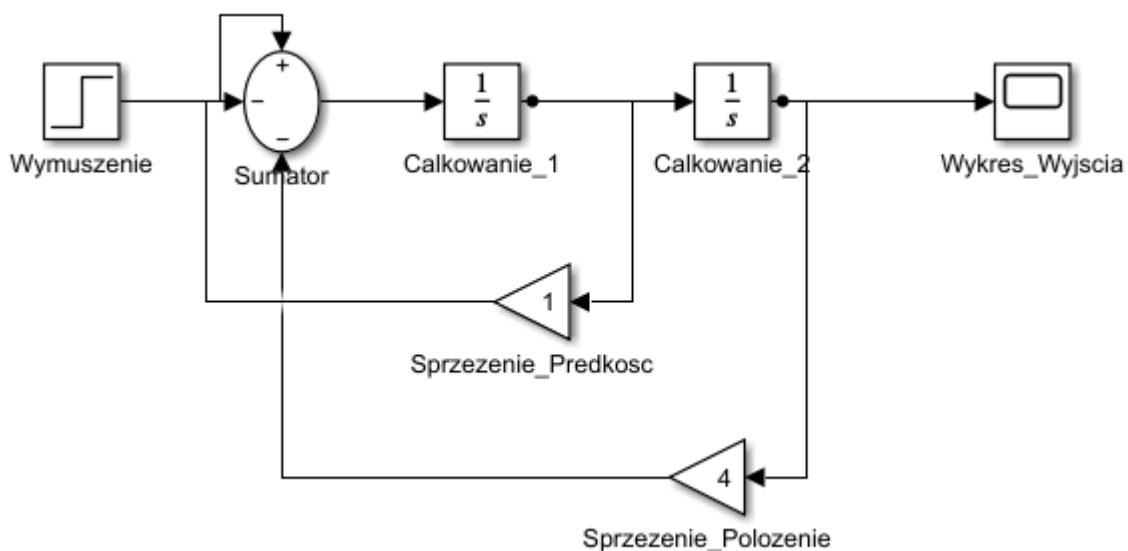
ĆWICZENIE 6 – Modelowanie w simulinku ($\ddot{y} + \dot{y} + 4y = 4$)

Analiza matematyczna stanu ustalonego (gdzie wszystkie pochodne się zerują) wskazuje, że równanie redukuje się do postaci $4y=4$, z czego wynika, że docelową wartością sygnału wyjściowego jest $y=1$. Po otwarciu bloku oscyloskopu (Scope) można dostrzec typową odpowiedź skokową z tłumieniem – układ jest tzw. układem niedotłumionym (z powodu niskiego współczynnika przy pierwszej pochodnej równego 1). Z tego względu sygnał oscyluje w stanie przejściowym, a na początku pojawia się przeregulowanie (sygnał "przestrzeliwuje" wartość 1), po czym drgania wygasają i po około 8 sekundach ustalają się na oczekiwanym poziomie.

KOD:

```
4 nazwa_pliku = 'Symulacja_Ukladu_Regulacji';
5 close_system(nazwa_pliku, 0);
6 new_system(nazwa_pliku);
7 open_system(nazwa_pliku);
8
9 % Dodanie bloków
10 add_block('simulink/Sources/Step', [nazwa_pliku, '/Wymuszenie'], 'Time', '0', 'Before', '0', 'After', '4', 'Position', [30 80 60 110]);
11 add_block('simulink/Math Operations/Sum', [nazwa_pliku, '/Sumator'], 'Inputs', '+--', 'Position', [110 75 140 115]);
12 add_block('simulink/Continuous/Integrator', [nazwa_pliku, '/Calkowanie_1'], 'Position', [190 80 220 110]);
13 add_block('simulink/Continuous/Integrator', [nazwa_pliku, '/Calkowanie_2'], 'Position', [290 80 320 110]);
14 add_block('simulink/Math Operations/Gain', [nazwa_pliku, '/Sprzezenie_Predkosc'], 'Gain', '1', 'Position', [210 160 240 190], 'BlockMirror', 'on');
15 add_block('simulink/Math Operations/Gain', [nazwa_pliku, '/Sprzezenie_Polozenie'], 'Gain', '4', 'Position', [280 230 310 260], 'BlockMirror', 'on');
16 add_block('simulink/Sinks/Scope', [nazwa_pliku, '/Wykres_Wyjscia'], 'Position', [410 80 440 110]);
17
18 % Łączenie bloków liniami sygnałowymi
19 add_line(nazwa_pliku, 'Wymuszenie/1', 'Sumator/1', 'autorouting', 'on');
20 add_line(nazwa_pliku, 'Sumator/1', 'Calkowanie_1/1', 'autorouting', 'on');
21 add_line(nazwa_pliku, 'Calkowanie_1/1', 'Calkowanie_2/1', 'autorouting', 'on');
22 add_line(nazwa_pliku, 'Calkowanie_2/1', 'Wykres_Wyjscia/1', 'autorouting', 'on');
23
24 % Linie sprzężenia zwrotnego
25 add_line(nazwa_pliku, 'Calkowanie_1/1', 'Sprzezenie_Predkosc/1', 'autorouting', 'on');
26 add_line(nazwa_pliku, 'Sprzezenie_Predkosc/1', 'Sumator/2', 'autorouting', 'on');
27 add_line(nazwa_pliku, 'Calkowanie_2/1', 'Sprzezenie_Polozenie/1', 'autorouting', 'on');
28 add_line(nazwa_pliku, 'Sprzezenie_Polozenie/1', 'Sumator/3', 'autorouting', 'on');
29
30 % Parametry i uruchomienie symulacji
31 set_param(nazwa_pliku, 'StopTime', '10');
32 sim(nazwa_pliku);
```

SIMULINK:



WYKRES:

